Testing

Two suites for two different jobs, and a third thing that is not a suite at all.

Command	What runs	Needs	Takes
`npm test`	The rules, as plain functions	nothing	~2 seconds
`npm run test:integration`	The API against a real Postgres	Docker	~30 seconds
`npm run test:all`	Both	Docker	~35 seconds
`npm run test:db:stop`	Removes the test database container	Docker	—
`npm run test:preview`	A walk through the customer app on the demo shop	the preview running	~2 seconds

478 tests today: 212 on the rules, 266 on the API. Both suites must be green before anything is pushed; the preview walk is run by hand when the demo shop is up.

The rules (`src/**/*.test.ts`)

Pure functions with no database: free slots and opening hours, how a visit was paid, session lengths, phone numbers, receipts, who may use which door — and since, the loyalty card's arithmetic, what a rating averages to, who belongs in which customer group, when a birthday falls in shop time, and what the insight screens count. Eighteen files, fast enough to keep running while you work (`npm run test:watch`).

These exist because the awkward parts of this system are arithmetic — a buffer around an appointment, a mixed payment that must add up, a clock that changes twice a year. Arithmetic is cheap to test and expensive to get wrong.

The API (`tests/*.test.ts`)

The route handlers, against a real Postgres in a container, with the session mocked to whichever role the test is about. Sixteen files, of which the ones worth knowing about are:

- **`booking-api.test.ts`** — booking, clashes, opening hours, buffers, time off, moving, cancelling, and who may use the shop's diary.

- `customer-api.test.ts` — signing up, what a customer may book, the notice period, the limit on waiting bookings, cancelling their own and not others', and the wall between customer and shop.
- `money-api.test.ts` — a visit rung up at the till through the drawer, the day journal and the period report: cash, card, InstaPay, mixed, tips, expenses, advances.
- `auth.test.ts` — signing in by email or phone, and how long a login lasts.
- `checkout-api.test.ts`, `queue-api.test.ts`, `diary-summary-api.test.ts` — ringing a booking up, the live queue, and the day at a glance.
- `barber-app.test.ts`, `barber-visibility.test.ts` — a barber's own day and earnings, and who the customer app is allowed to show.
- `offers-api.test.ts`, `push-api.test.ts`, `reviews-api.test.ts` — offers, notifications and reminders, and reviews: who may leave one, one per visit, and the owner's power to hide but not delete.
- `loyalty-api.test.ts` — stamps counted from paid visits only, a card that cannot be cashed in twice, and the proof that giving a reward moves no money.
- `insights-api.test.ts` — the owner's read of the shop: the window, who may ask, and the proof that a long-standing customer is never counted as a new face.
- `segments-api.test.ts` — the groups, and the guardrails on writing to one: who may send, once every twelve hours, and only to people who asked for notifications.
- `referrals-api.test.ts` — a code that survives being written down badly, a referral earned only by a friend who actually came in, a reward given once, and a birthday greeting that goes out once a year.

A mock cannot tell you that two people took the same slot, or that the drawer and the report disagree. These can.

The database is a throwaway container (`escape-test-db` , port 55499), started on the first run and reused afterwards. Every table is emptied between tests, and the connection string is built in `tests/db.ts` — the shop's real database is never reachable from a test.

The walk through the preview (`tests/manual`)

`npm run test:preview` walks the customer app end to end against the demo shop on your own machine: the profile and its full loyalty card, the price list, free times on a real day, a booking that it then cancels, a review, the referral code and the notification key. It prints what it found rather than only passing.

It is kept out of the normal suite on purpose. The preview is not always up, and a green suite must mean the code is sound rather than that somebody happened to have Docker running.

Writing another one

- A new rule starts as a function in `src/lib` with a unit test, before any screen.
- A new endpoint gets an integration test covering what it does **and** who is refused. Every route was written with both, and one of those refusal tests is what caught a customer being able to open the shop's own appointments screen.
- Reach for real rows over mocks. The bugs live in what the database actually did.

What is not covered yet

- The screens themselves: no browser tests. Checked by hand for now.
- The Excel and PDF exports beyond the receipt text.
- The warehouse and payroll screens, which predate this suite and are covered only through the money tests they feed.